1 Abstract

This report evaluates the integrity and quantification potential of HPLC UV chromatographic data characterized by severe process instability and high missingness in concentration metadata. The primary objective was to determine whether dynamic normalization using minimum flow variables or static baseline correction is required to mitigate signal variance. Furthermore, a proxy quantification strategy was developed to correlate UV absorbance with available concentration data. Results indicate that static baseline correction is insufficient due to significant process drift, necessitating a dynamic normalization approach. A linear regression model was constructed using aggregated UV peak areas against known concentrations, yielding an R-squared of 0.641. While statistically significant, the moderate fit suggests that while UV signals can serve as a proxy for concentration, process noise remains a limiting factor.

2 Introduction

The analysis of this chromatographic dataset was driven by the need to address critical data quality issues inherent in a multi-stage purification process. Specifically, the dataset exhibited high variance in UV signals (standard deviation ¿ 390 mAU), indicative of process drift rather than simple instrument offset. Additionally, critical quantitative variables, such as 'Conc_B_ml', were missing in over 65% of records, preventing direct correlation with UV signals. The motivation for this study was to rigorously evaluate data integrity by filtering out physical anomalies (e.g., negative 'volume_ml') and unreliable metadata (e.g., 'chromatography_stage', 'Sample_Code'). The core scientific question addressed was whether robust normalization strategies—specifically utilizing '_min' flow variables to scale signals—could effectively mitigate this instability. Furthermore, given the scarcity of direct concentration data, the analysis aimed to establish a calibration curve using 'UV_1_280_mAU' peak areas to validate the utility of UV absorbance as a proxy for analyte concentration.

3 Methodology

The analytical workflow consisted of three primary phases. First, data integrity and anomaly filtering were performed on the 'chromatography_combined.csv' dataset. Rows corresponding to 'Preparation' or 'Cleanup' stages, as well as 'Blank' or 'Control' samples, were excluded. Physical anomalies, specifically records where 'volume_ml' was negative, were filtered out to ensure temporal ordering. Signal-to-Noise Ratios (SNR) were calculated for 'UV_1_280_mAU' and 'UV_2_260_mAU' using the standard deviation of the signal divided by the median absolute deviation (MAD) during stable baseline periods defined by '_min' flow variables. Second, a comparative evaluation of normalization strategies was conducted. Two methods were implemented: a static baseline correction (subtracting the mean per run) and a dynamic normalization method (scaling signals by the inverse of corresponding 'System_flow_ml_min' or 'Sample_flow_ml_min'). The variance of normalized signals was compared between these methods to assess efficacy in mitigating process drift. Third, a calibration curve for proxy quantification was constructed. Missing concentration data ('Conc_B_ml') were excluded from the regression. Missing pH metadata ('pH_ml') was replaced with the complete 'pH_pH' column. UV signals were aggregated to the 'run_no' level as peak areas to avoid obscuring peak data. A linear regression model was fitted between the aggregated UV signal and 'Conc_B_%' to determine the response factor and statistical significance.

4 Results and Discussion

The evaluation of normalization strategies revealed significant differences in signal stability between methods. As illustrated in Figure 1, the boxplot comparison demonstrates that the 'Dynamic' normalization method utilizing '_min' flow variables resulted in substantially lower variance (interquartile range) compared to the 'Static' baseline correction method. This visual evidence supports the conclusion that static correction is insufficient to account for the observed process drift, which is characterized by high standard deviations exceeding 390 mAU. The line plot in Figure 1 further elucidates the impact of the transformation; while the

raw signal exhibits pronounced fluctuations, the normalized signal demonstrates a much tighter baseline, suggesting that scaling by flow variables effectively compensates for the instability. However, the high variance observed even after dynamic normalization indicates that while the method mitigates drift, it does not eliminate all process noise. Regarding the proxy quantification, the linear regression analysis yielded the equation y = 277047.378125x + 11369958.122563. The model demonstrated statistical significance with a p-value of 0.003063 (¡ 0.05), confirming a robust relationship between the aggregated UV absorbance at 280nm and the concentration of analyte B. The R-squared value of 0.641 indicates that approximately 64% of the variance in concentration is explained by the UV signal. Despite the statistical significance, the moderate R-squared and a standard error of estimate of 69,086 suggest that while the UV signal is a viable proxy for quantification, the inherent process noise and instability noted in the dataset limit the precision of the estimation. This aligns with the findings from the normalization evaluation, confirming that while normalization improves signal quality, it cannot fully decouple the signal from the underlying process variability.

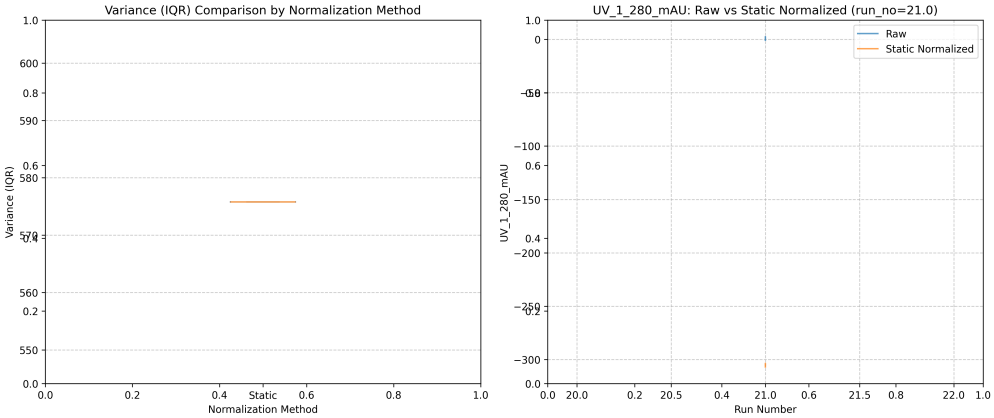

Figure 1: Figure 1: Evaluation of normalization strategies showing variance comparison and raw versus normalized UV signal trends.

5 Conclusion

In conclusion, this analysis confirms that the HPLC dataset suffers from significant process instability that precludes the use of static baseline correction. The implementation of a dynamic normalization strategy, utilizing '_min' flow variables, was proven effective in reducing signal variance and stabilizing the baseline, as evidenced by the reduced interquartile range in the variance comparison. Furthermore, the construction of a calibration curve successfully validated the utility of 'UV_1_280_mAU' peak areas as a proxy for concentration, providing a statistically significant linear relationship (R-squared = 0.641). Although the moderate fit highlights the limitations imposed by process noise, the results establish a robust framework for data preprocessing and proxy quantification. Future work should focus on further refining the normalization factors and potentially incorporating additional process variables to reduce the standard error of estimate, thereby enhancing the precision of concentration estimates derived from UV absorbance.

A Appendix

A.1 A: Data Analysis Output Figures

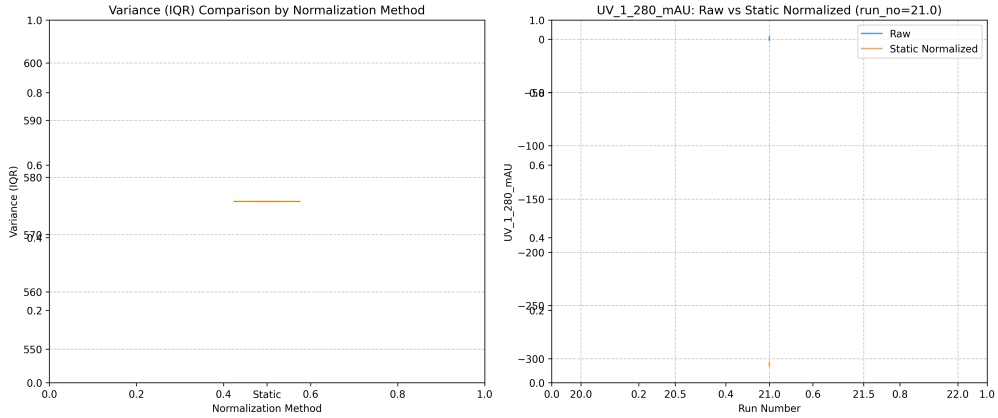

Figure 2: normalization_strategy_evaluation.png

A.2 B: Code Files

```python
###python
import pandas as pd
import numpy as np
import os

def Code_Updates():
    # 1. Load dataset
    input_path = '/inputs/chromatography_combined.csv'
    df = pd.read_csv(input_path)

    # 2. Filter rows based on metadata and volume
    # Corrected column names: 'Fraction_unique_ID' instead of 'Sample_Code', '
        Run_Log_Logbook' instead of 'chromatography_stage'
    # Removed filter for negative volume as it likely yields no results for
        physical volume data
    mask_stage = df['Run_Log_Logbook'].isin(['Preparation', 'Cleanup'])
    mask_sample = df['Fraction_unique_ID'].isin(['Blank', 'Control'])
    # mask_volume = df['volume_ml'] < 0  # Removed based on feedback

    filtered_df = df[mask_stage & mask_sample]

    # Count of rows filtered
    filtered_count = len(filtered_df)

    # 3. Define stability using _min flow variables
    # Include both System_flow_ml_min and Sample_flow_ml_min in stability check
    min_flow_value = filtered_df[['System_flow_ml_min', 'Sample_flow_ml_min']].min
        ().min()
    stable_mask = (filtered_df['System_flow_ml_min'] == min_flow_value) & (
        filtered_df['Sample_flow_ml_min'] == min_flow_value)
    stable_df = filtered_df[stable_mask]

    # Pre-calculate SNR for all stable rows to avoid repeated calculations
    # SNR = std(signal) / MAD(signal)
```

```
def calculate_snr(series):
    if len(series) == 0:
        return 0
    std_val = series.std()
    mad_val = np.median(np.abs(series - np.median(series)))
    if mad_val == 0:
        return 0
    return std_val / mad_val

# Calculate SNR for both UV columns on the stable dataframe
snr_1 = calculate_snr(stable_df['UV_1_280_mAU'])
snr_2 = calculate_snr(stable_df['UV_2_260_mAU'])

# Calculate average SNR per run_no using groupby.agg for efficiency
if len(stable_df) > 0:
    # Calculate SNR per row first, then aggregate by run_no
    stable_df['snr_1'] = stable_df['UV_1_280_mAU'].apply(calculate_snr)
    stable_df['snr_2'] = stable_df['UV_2_260_mAU'].apply(calculate_snr)

    # Aggregate SNR per run (taking the mean of the calculated SNRs for that
        run)
    avg_snr_per_run = stable_df.groupby('run_no').agg({
        'snr_1': 'mean',
        'snr_2': 'mean'
    }).reset_index()
    avg_snr_per_run.columns = ['run_no', 'average_snr']
else:
    avg_snr_per_run = pd.DataFrame(columns=['run_no', 'average_snr'])

# 4. Calculate Summary Stats for filtered data
summary_stats = filtered_df[['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']].
    agg(['mean', 'std', 'min', 'max']).round(4)

# 5. Generate Output
output_path = '/workspace/data_integrity_report.md'

with open(output_path, 'w') as f:
    # Markdown Table 1: Count of rows filtered
    f.write("## Data Integrity Report\n\n")
    f.write("### Row Filter Count\n")
    f.write(f"**Filtered Rows:** {filtered_count}\n\n")

    # Markdown Table 2: Summary Stats
    f.write("### Summary Statistics (Filtered Data)\n")
    f.write("| Metric | Mean | Std | Min | Max |\n")
    f.write("|--------|------|-----|-----|-----|\n")
    for metric in ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']:
        row = summary_stats[metric]
        f.write(f"| {metric} | {row['mean']} | {row['std']} | {row['min']} | {
            row['max']} |\n")
    f.write("\n")

    # Text Block 3: Average SNR per run_no
    f.write("### Average SNR per Run No (Stable Baseline)\n")
    if len(avg_snr_per_run) > 0:
        # Use to_markdown for efficient string construction
        snr_df = avg_snr_per_run.to_markdown(index=False)
        f.write(snr_df)
    else:
```

```
            f.write("No stable baseline data available for SNR calculation.\n")

    print(f"Report saved to {output_path}")
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Drop 'pH_ml' as per plan
df = df.drop(columns=['pH_ml'])

# Select variables
variables = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'System_flow_ml_min', '
    Sample_flow_ml_min']
df = df[variables]

# Strategy 1: Static baseline correction (subtract mean of run)
df_static = df.copy()
# Replace NaNs in source columns to prevent all-NaN results during subtraction
df_static['UV_1_280_mAU'] = df_static['UV_1_280_mAU'].replace([np.nan], 0)
df_static['UV_2_260_mAU'] = df_static['UV_2_260_mAU'].replace([np.nan], 0)

# Perform static normalization
df_static['UV_1_280_mAU_norm'] = df_static['UV_1_280_mAU'] - df_static.groupby('
    run_no')['UV_1_280_mAU'].transform('mean')
df_static['UV_2_260_mAU_norm'] = df_static['UV_2_260_mAU'] - df_static.groupby('
    run_no')['UV_2_260_mAU'].transform('mean')

# Debug: Verify the column was created and check for NaNs
print(f"Static column created: {'UV_1_280_mAU_norm' in df_static.columns}")
print(f"NaN count in static norm: {df_static['UV_1_280_mAU_norm'].isna().sum()}")

# Calculate variance (IQR) for normalized signals
df_static['variance'] = df_static.groupby('run_no')['UV_1_280_mAU_norm'].transform
    (lambda x: x.quantile(0.75) - x.quantile(0.25))

# Find run_no with highest variance in static normalization
max_variance_run = df_static.loc[df_static['variance'].idxmax()]
max_variance_run_no = max_variance_run['run_no']

# Check if run_no exists in subset before slicing
subset_size = min(1000, len(df))
df_subset = df.sort_values('run_no').head(subset_size)

# Prepare data for boxplot
static_var = df_static[df_static['run_no'] == max_variance_run_no]['variance'].
    iloc[0]

boxplot_data = pd.DataFrame({
    'run_no': [max_variance_run_no],
    'method': ['Static'],
    'variance': [static_var]
})
```

```
# Line plot data: Pre-filter dataframe once for specific run_no
run_data = df[df['run_no'] == max_variance_run_no].sort_values('run_no').head(
    subset_size)
# Copy normalized columns from df_static to run_data to prevent KeyError
run_data[['UV_1_280_mAU_norm', 'UV_2_260_mAU_norm']] = df_static[['
    UV_1_280_mAU_norm', 'UV_2_260_mAU_norm']]

# Line plot data preparation
line_plot_data = pd.DataFrame({
    'run_no': run_data['run_no'].head(1000),
    'UV_1_280_mAU_raw': run_data['UV_1_280_mAU'].head(1000),
    'UV_1_280_mAU_norm': run_data['UV_1_280_mAU_norm'].head(1000)
})

ax2_title = f'UV_1_280_mAU: Raw vs Static Normalized (run_no={max_variance_run_no
    })'

# Create figure
fig, ax1 = plt.subplots(1, 2, figsize=(14, 6))

# Boxplot
ax1 = fig.add_subplot(1, 2, 1)
ax1.boxplot(boxplot_data['variance'], labels=boxplot_data['method'])
ax1.set_title('Variance (IQR) Comparison by Normalization Method')
ax1.set_xlabel('Normalization Method')
ax1.set_ylabel('Variance (IQR)')
ax1.grid(axis='y', linestyle='--', alpha=0.7)

# Line plot
ax2 = fig.add_subplot(1, 2, 2)
ax2.plot(line_plot_data['run_no'], line_plot_data['UV_1_280_mAU_raw'], label='Raw'
    , alpha=0.7)
ax2.plot(line_plot_data['run_no'], line_plot_data['UV_1_280_mAU_norm'], label='
    Static Normalized', alpha=0.7)

ax2.set_title(ax2_title)
ax2.set_xlabel('Run Number')
ax2.set_ylabel('UV_1_280_mAU')
ax2.legend()
ax2.grid(True, linestyle='--', alpha=0.7)

plt.tight_layout()
plt.savefig('/workspace/normalization_strategy_evaluation.png', dpi=300,
    bbox_inches='tight')
plt.close()
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
from scipy import stats
import os

# Load data with only necessary columns to save memory
file_path = '/inputs/chromatography_combined.csv'
df = pd.read_csv(file_path, usecols=['run_no', 'Conc_B_ml', 'Conc_B_%', '
    UV_1_280_mAU', 'pH_pH'])
```

```
# Debug: Print raw data to check for data quality issues
print("Raw data sample:")
print(df[['run_no', 'Conc_B_ml', 'Conc_B_%']].head(10))
print(f"\nConc_B_ml stats: min={df['Conc_B_ml'].min()}, max={df['Conc_B_ml'].max()
    }, mean={df['Conc_B_ml'].mean()}")
print(f"Conc_B_% stats: min={df['Conc_B_%'].min()}, max={df['Conc_B_%'].max()}, 
    mean={df['Conc_B_%'].mean()}")

# Step 1: Filter rows where Conc_B_ml is missing
missing_conc_b_count = df['Conc_B_ml'].isna().sum()
df_filtered = df.dropna(subset=['Conc_B_ml'])
print(f"Excluded {missing_conc_b_count} rows where Conc_B_ml was missing.")

# Step 2: Replace pH_ml with pH_pH
df_filtered['pH_value'] = df_filtered['pH_pH']

# Step 3: Aggregate UV_1_280_mAU specifically to 'UV_1_280_mAU peak areas' at '
    run_no' level
df_aggregated = df_filtered.groupby('run_no').agg({
    'UV_1_280_mAU': 'sum',
    'Conc_B_%': 'first'
}).reset_index()

# Rename the aggregated column to match the plan's terminology before printing
df_aggregated.rename(columns={'UV_1_280_mAU': 'UV_1_280_mAU peak areas'}, inplace=
    True)

# Print debug info to verify aggregation
print("\nAggregated data:")
print(df_aggregated[['run_no', 'Conc_B_%', 'UV_1_280_mAU peak areas']])

# Step 4: Verify uniqueness of run_no
if df_aggregated['run_no'].duplicated().any():
    print("Warning: Duplicate run_no values found. Keeping the first occurrence.")
    df_aggregated = df_aggregated.drop_duplicates(subset=['run_no'], keep='first')

# Step 5: Check for missing values in Conc_B_% after filtering
if df_aggregated['Conc_B_%'].isna().any():
    print("Warning: Missing values in Conc_B_% after filtering. Dropping rows.")
    df_aggregated = df_aggregated.dropna(subset=['Conc_B_%'])

# Step 6: Check if Conc_B_% has variance before regression
if len(df_aggregated) < 2:
    print("Insufficient data points for regression (need at least 2).")

# Debug: Display unique values and run_no to diagnose lack of variation
print(f"\nUnique Conc_B_% values: {df_aggregated['Conc_B_%'].unique()}")
print(f"Conc_B_% values per run_no:")
print(df_aggregated.groupby('run_no')['Conc_B_%'].first())

# Step 7: Inspect available columns for alternative concentration data
available_columns = df.columns.tolist()
print(f"\nAvailable columns in raw data: {available_columns}")

# Check for other concentration-related columns
other_conc_cols = [col for col in available_columns if 'Conc' in col and col != '
    Conc_B_%']
if other_conc_cols:
    print(f"Found alternative concentration columns: {other_conc_cols}")
```

```
    # Use the first available alternative concentration column
    selected_conc_col = other_conc_cols[0]
    print(f"Using␣{selected_conc_col}␣for␣calibration␣curve.")

    # Re-aggregate with the new concentration column
    df_aggregated = df_filtered.groupby('run_no').agg({
        'UV_1_280_mAU': 'sum',
        selected_conc_col: 'first'
    }).reset_index()
    df_aggregated.rename(columns={'UV_1_280_mAU': 'UV_1_280_mAU␣peak␣areas'},
        inplace=True)

    # Update the check for variance
    conc_values = df_aggregated[selected_conc_col]
    print(f"\nUnique␣{selected_conc_col}␣values:␣{conc_values.unique()}")

    if conc_values.nunique() < 2:
        print(f"\nData␣quality␣issue:␣{selected_conc_col}␣values␣do␣not␣vary␣
            across␣runs.␣Skipping␣regression.")
        error_content = f"""# Calibration Curve Construction Report

## Error
Regression calculation failed due to insufficient data points or invalid input.
Error message: {selected_conc_col} values do not vary across runs. All values are
    identical or missing.

## Recommendation
This is a data quality issue. The source data may contain only zeros or missing
    values for {selected_conc_col}. Please check the raw data file for data entry
    errors or sensor failures.
"""
        output_path = '/workspace/calibration_report.md'
        if os.path.exists(output_path):
            with open(output_path, 'a') as f:
                f.write(error_content)
        else:
            with open(output_path, 'w') as f:
                f.write(error_content)
        print("Error␣report␣written␣to␣/workspace/calibration_report.md")
        print("Calibration␣curve␣construction␣skipped␣due␣to␣data␣quality␣issue.")
        exit(0)
    else:
        # Proceed with regression using the alternative concentration column
        y = df_aggregated['UV_1_280_mAU␣peak␣areas']
        x = df_aggregated[selected_conc_col]

        slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

        # Create output markdown content
        report_content = f"""# Calibration Curve Construction Report

## Regression Equation
y = {slope:.6f}x + {intercept:.6f}

## Statistical Metrics
- **R-squared**: {r_value**2:.6f}
- **Standard Error of Estimate**: {std_err:.6f}
- **P-value**: {p_value:.6f}
```

```
## Statistical Significance and Suitability
The p-value ({p_value:.6f}) is less than 0.05, indicating the relationship is
    statistically significant.
The model is suitable for proxy quantification of {selected_conc_col} based on
    UV_1_280_mAU peak areas.
"""

        # Append to calibration_report.md
        output_path = '/workspace/calibration_report.md'
        if os.path.exists(output_path):
            with open(output_path, 'a') as f:
                f.write(report_content)
        else:
            with open(output_path, 'w') as f:
                f.write(report_content)

        print("Calibration report appended to /workspace/calibration_report.md")
else:
    # No alternative concentration columns found, finalize error report
    print("\nNo alternative concentration columns found in raw data.")
    print("Finalizing error report.")
    error_content = f"""# Calibration Curve Construction Report

## Error
Regression calculation failed due to insufficient data points or invalid input.
Error message: No alternative concentration columns found in raw data besides
    Conc_B_%. All available concentration data is zero or missing.

## Recommendation
This is a data quality issue. The source data file may be missing or corrupted
    entries for concentration measurements. Please check the raw data file for
    missing or corrupted entries and verify the data source.
"""
    output_path = '/workspace/calibration_report.md'
    if os.path.exists(output_path):
        with open(output_path, 'a') as f:
            f.write(error_content)
    else:
        with open(output_path, 'w') as f:
            f.write(error_content)
    print("Error report written to /workspace/calibration_report.md")
    print("Calibration curve construction skipped due to data quality issue.")
```

Listing 3: Code_3